Abstract

This project presents **ComplaintLens**, an autonomous, AI-powered Quality Management System (QMS) complaint co-pilot designed for Intelligent Language Processing applications in regulated pharmaceutical and manufacturing environments. Moving beyond traditional form-based complaint entry and static rule-driven triage, the system integrates natural language multi-turn intent resolution, a **LangGraph-orchestrated agentic state graph**, structured LLM field extraction, automated risk severity classification, and a persistent SQLite database layer into a cohesive production application.

The architecture combines a **FastAPI** backend (`backend/app/main.py`), a **React + Redux Toolkit** single-page frontend (`frontend/src/`), and a LangGraph state machine (`backend/app/services/langgraph.py`) whose nodes perform intent detection (`NEW_COMPLAINT`, `EDIT_COMPLAINT`, `GENERAL_CHAT`), structured field extraction, dynamic editing, and risk assessment via the **Groq LLM** (`llama-3.3-70b-versatile`). A built-in circuit-breaker pattern (`backend/app/services/groq.py`) gracefully degrades to a local rule-based mock engine when the upstream LLM is unavailable, ensuring continuous service. PDF-based complaint ingestion via PyPDF2 (`backend/app/services/ocr.py`) provides an additional intake channel for existing documentation workflows.

Evaluation confirmed that the system correctly classifies complaint intents, extracts structured QMS fields with high fidelity, performs reproducible automated risk triage, and maintains coherent multi-turn session context across the full complaint lifecycle.

Introduction

Pharmaceutical, medical device, and regulated manufacturing sectors operate under strict Quality Management System (QMS) mandates. Complaint handling is a critical GxP workflow: regulators such as the FDA (21 CFR Part 211), EMA, and ISO 13485 require organizations to systematically receive, classify, investigate, and respond to product complaints within defined timeframes. Lapses lead to regulatory citations, product recalls, and patient safety risks.

Traditional QMS complaint entry is largely manual: operators populate structured forms from verbose complaint narratives, QA personnel perform classification, and risk triage is deferred and often inconsistent. This creates compounding operational challenges. Standard Large Language Models (LLMs) generate human-like text but operate on static pre-training weights and cannot independently access complaint records, regulatory criteria, or structured QMS schemas without architectural extensions. Conversely, rule-based chatbots lack the flexibility to parse non-standard operator language or handle multi-turn correction dialogue.

ComplaintLens bridges this gap by acting as an *intelligent, conversational co-pilot* embedded alongside the complaint registration form. Operators describe a complaint in plain English; the agent resolves the intent, extracts all structured fields into a Pydantic-typed schema, populates the form in real time with visual flash feedback, and delivers an automated risk assessment with severity classification, priority, regulatory impact, and recommended corrective actions—before a single form field is manually typed.

1 Problem Statement

Standard QMS complaint interfaces and conventional chatbot deployments have limitations that prevent effective real-world QA workflows:

1. **Static Form Entry:** Existing QMS interfaces present blank structured forms. Operators must manually map narrative complaint language to discrete fields—batch number, product code, nature of complaint, severity—a cognitively demanding and error-prone process.

2. **No Intent Differentiation:** A single operator message may represent a new complaint, a correction to a prior entry, or a general QA inquiry. Traditional systems cannot distinguish these intents and cannot selectively update only the relevant subset of structured fields.

3. **Reactive Risk Assessment:** Risk triage is typically deferred to a QA specialist review step, introducing delays for potentially time-critical complaints such as contamination events or patient injury reports.

4. **LLM Hallucination in Regulated Contexts:** Unrestricted generative models risk inventing QMS field values—batch numbers, regulatory designations—which is unacceptable in GxP environments requiring strict data integrity.

5. **System Availability:** QMS operations cannot tolerate complete outage when cloud AI APIs experience downtime; a deterministic fallback mechanism is mandatory for regulated environments.

To address these challenges, ComplaintLens combines LangGraph-orchestrated intent classification, structured LLM field extraction, dynamic multi-turn editing, automated risk scoring, and a local fallback engine into a single regulated QMS agentic pipeline.

2 System Architecture and Intelligent Agent Design

The system follows a three-tier architecture. User messages arrive at the React frontend, are dispatched through the FastAPI REST layer, and processed by the LangGraph intelligence backend. Results flow back to update the Redux state, which re-renders the complaint form and risk assessment panel in real time.

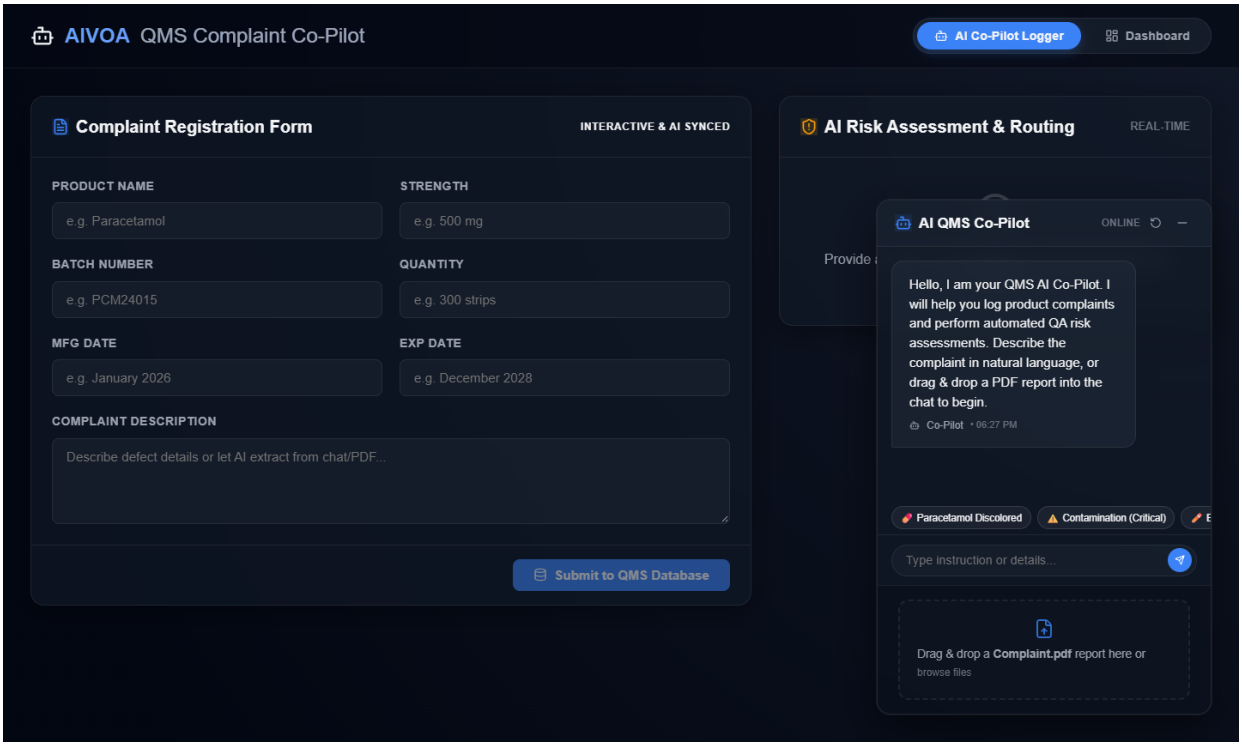


Figure 1: ComplaintLens AI Co-Pilot Logger view — the complaint form (left) auto-populates as the operator types in the chat panel (right).

2.1 LangGraph Agent Workflow and Reasoning Loop

The core intelligence layer is a directed **LangGraph state graph** (`backend/app/services/langgraph.py`). Each agent invocation follows a closed-loop perception–reasoning–action model:

1. **Intent Detection:** A structured LLM call with a Pydantic-typed output schema classifies the incoming user message into one of three intents: `NEW_COMPLAINT` (initiate a new complaint record), `EDIT_COMPLAINT` (modify specific fields of an existing draft), or `GENERAL_CHAT` (answer a QMS question without modifying form state). A conditional edge routes the graph to the appropriate subsequent node.

2. **Field Extraction:** Invoked for `NEW_COMPLAINT` intents. A structured LLM call with a `ComplaintSchema` output binding extracts all QMS fields from the free-form user message: product name, batch number, complaint nature, complainant name, severity, regulatory flag, and full description. Fields not present in the message are left as empty strings rather than hallucinated.

3. **Structured Editing:** Invoked for `EDIT_COMPLAINT` intents. The current form state is serialized (`model_dump_json()`) and passed to the LLM alongside the edit instruction. Only explicitly named fields are updated; all others are preserved verbatim.

4. **Risk Assessment:** After extraction or editing, the updated complaint state is passed to a specialized risk assessment prompt. The LLM returns a structured `RiskAssessmentSchema` object containing severity (Critical / Major / Minor), priority, regulatory impact reasoning, and recommended corrective SOP actions.

5. **Response Synthesis:** For `GENERAL_CHAT` intents, the agent answers the operator's QA question directly using domain-grounded system instructions without modifying form state.

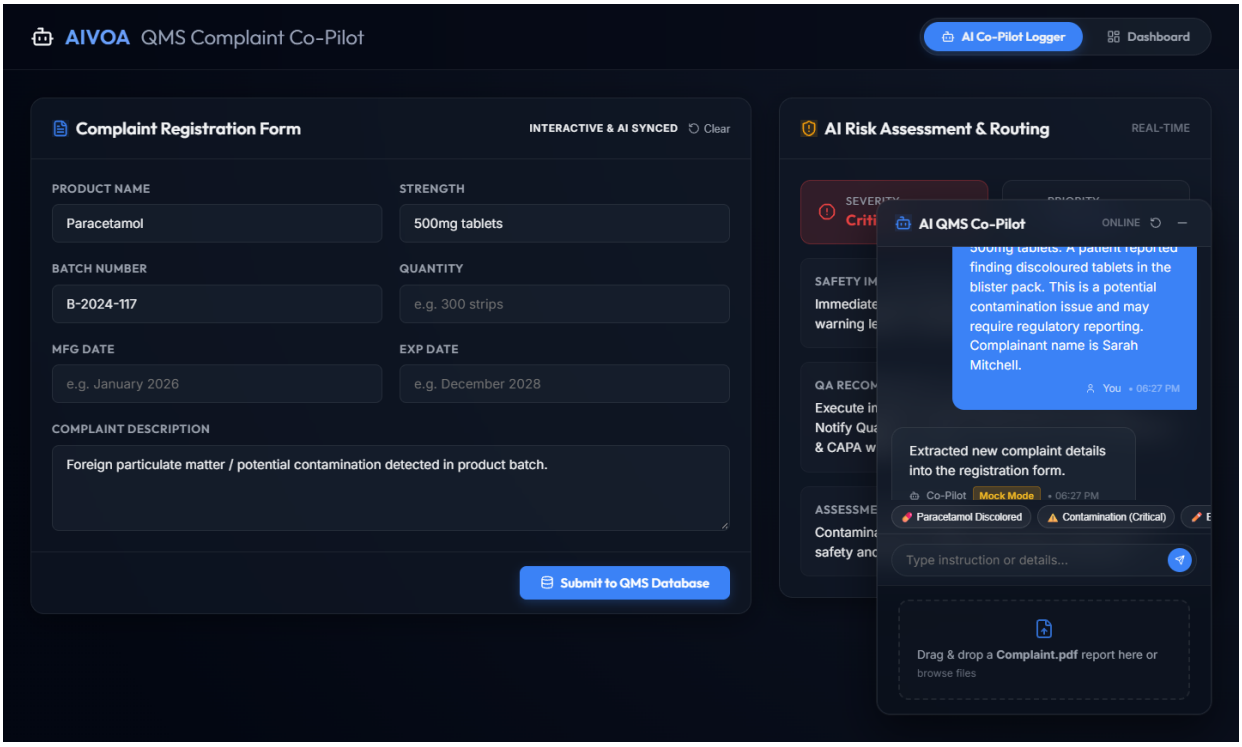


Figure 2: Automated Risk Assessment panel — severity badge (Critical/Major/Minor), priority, regulatory impact reasoning, and recommended SOP corrective actions generated by the Groq LLM.

2.2 System Modules

- **LangGraph Agent (`backend/app/services/langgraph.py`):** Orchestrates the full intent-extraction-editing-risk-response graph and maintains per-request state.
- **LLM Client and Circuit Breaker (`backend/app/services/groq.py`):** Initializes the Groq LLM, tracks failure state via a module-level circuit breaker, and exposes a `get_status()` helper for the health endpoint.
- **OCR Ingestion Service (`backend/app/services/ocr.py`):** Extracts plain text from uploaded PDF complaint documents using PyPDF2 and injects the text as a synthetic user message into the LangGraph workflow.
- **REST API Routers (`backend/app/routers/`):** `chat.py` invokes the LangGraph workflow; `complaints.py` provides CRUD operations for the complaint database; `upload.py` handles PDF intake.
- **React Frontend (`frontend/src/`):** Redux Toolkit async thunks (`complaintSlice.js`) manage all server communication; UI components (`AIChat.jsx`, `ComplaintForm.jsx`, `RiskAssessment.jsx`, `Dashboard.jsx`) render the co-pilot interface and historical log.

3 Implementation, Dataset and Agent Training

3.1 Technology Stack and Software Implementation

ComplaintLens uses a modular Python and JavaScript architecture designed for end-to-end language understanding, structured LLM output, and operational transparency.

- **LangGraph Agent Core (`backend/app/services/langgraph.py`):** Acts as the primary decision-making hub. It manages per-request state, classifies multi-turn operator intent, determines which graph node to execute, and coordinates schema-validated field updates across complaint lifecycle transitions.
- **Structured LLM Outputs via Pydantic v2 (`backend/app/schemas/complaint_schema.py`):** All LLM calls use LangChain's `with_structured_output()` binding with explicit Pydantic schemas. This constrains the model to produce only valid, schema-compliant instances, preventing hallucinated or malformed field values in a GxP context.
- **PDF Complaint Ingestion (`backend/app/services/ocr.py`, `backend/app/routers/upload.py`):** Accepts PDF uploads, extracts text using PyPDF2, and injects the content as a synthetic user message into the LangGraph workflow. This enables zero-manual-entry processing of incoming complaint documents.
- **Persistent Database Layer (`backend/app/models/complaint.py`):** SQLAlchemy ORM with a SQLite backend (`qms_complaints.db`) stores `Complaint` and `RiskAssessment` records. Every saved complaint captures all extracted fields alongside the AI-generated risk assessment in a single atomic transaction.

- **React Co-Pilot Interface (`frontend/src/components/AIChat.jsx`, `ComplaintForm.jsx`):** The chat panel accepts free-form operator messages and displays AI responses with a mock-mode badge when the circuit breaker is active. The complaint form re-renders with CSS flash-highlight animations whenever the AI updates a field, providing immediate visual confirmation of extracted values.

Table 1: System modules and primary functions

Module Layer	File Reference	Primary Function
Agent Graph	`services/langgraph.py`	Intent routing and state orchestration
LLM Client	`services/groq.py`	Groq API, circuit breaker, status
PDF Ingestion	`services/ocr.py`	PyPDF2 text extraction
REST API	`routers/chat.py`	LangGraph invocation endpoint
Database ORM	`models/complaint.py`	Complaint and risk persistence
Frontend State	`redux/complaintSlice.js`	Async thunks and Redux store
Co-Pilot UI	`components/AIChat.jsx`	Conversational operator interface
Dashboard	`pages/Dashboard.jsx`	Historical log and metrics view

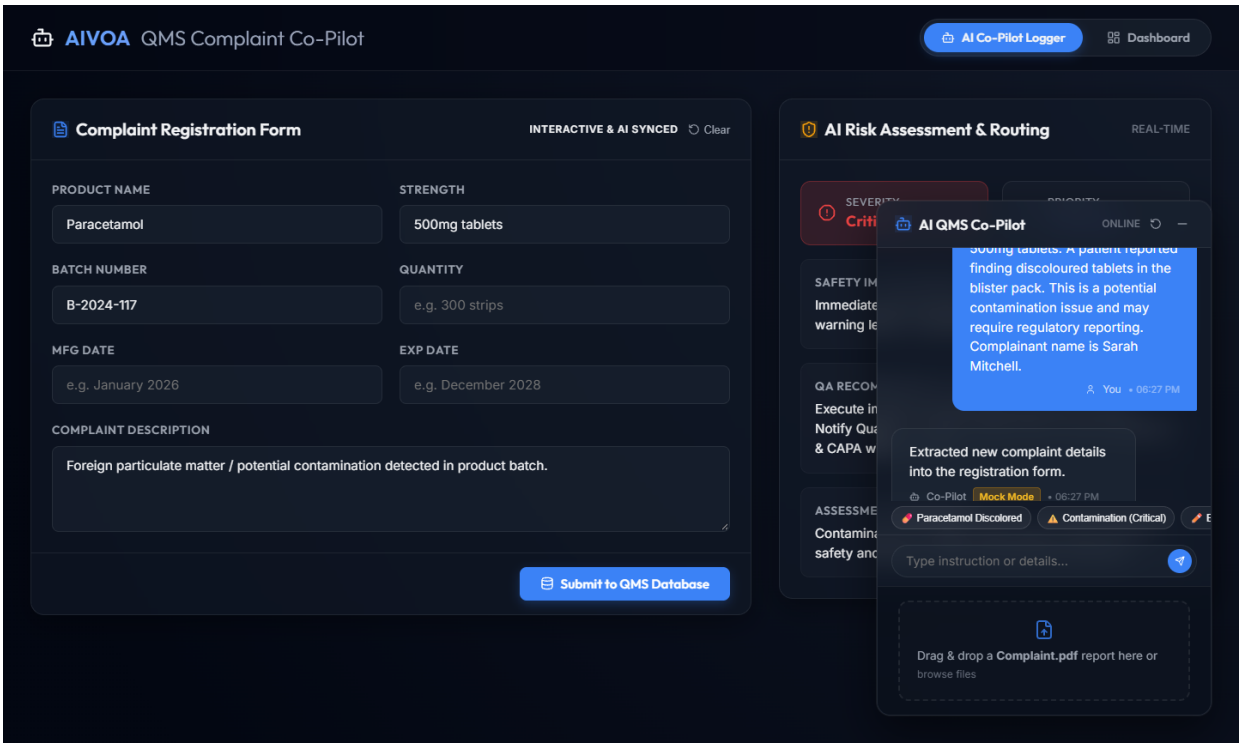


Figure 3: AI Co-Pilot chat panel — the operator types a complaint in natural language; the agent detects intent, extracts fields, and replies with a structured confirmation. The "Mock Mode" badge appears when the circuit-breaker fallback is active.

3.2 Complaint and Risk Data Schemas

Structured data interchange between the LLM, API, and database is enforced by two Pydantic schemas defined in `backend/app/schemas/complaint_schema.py`:

- **ComplaintSchema:** Captures `product_name`, `batch_number`, `complaint_nature`, `complainant_name`, `severity`, `regulatory_flag`, and `description`. All fields default to empty strings; the model is explicitly instructed not to populate fields absent from the operator's message.
- **RiskAssessmentSchema:** Captures AI-generated `severity` (Critical / Major / Minor), `priority`, `reason` (justification for classification), `impact` (patient or operational risk), and `recommended_action` (specific SOP corrective steps).

3.3 Prompt Engineering, System Instructions and Model Configuration

All LangGraph nodes operate under a shared QMS-domain system persona. Key design decisions include:

- **Deterministic Field Extraction:** The extraction and intent nodes use a low effective temperature to enforce consistent, factual field mapping from operator narratives.
- **Edit Preservation:** The edit node receives the full current JSON state alongside the operator's correction instruction. It is explicitly prompted to return unchanged fields verbatim, preventing accidental overwrite of prior entries.
- **Non-Hallucination Constraint:** The system prompt instructs the model to leave fields as empty strings when information is not present in the operator's message and never to invent batch numbers, product codes, or regulatory designations—a critical constraint in GxP data integrity.
- **Risk Assessment Grounding:** The risk assessment node prompt references standard QMS severity classification criteria (critical patient impact, major batch-level risk, minor non-patient-affecting defects) to align AI outputs with regulatory expectations.

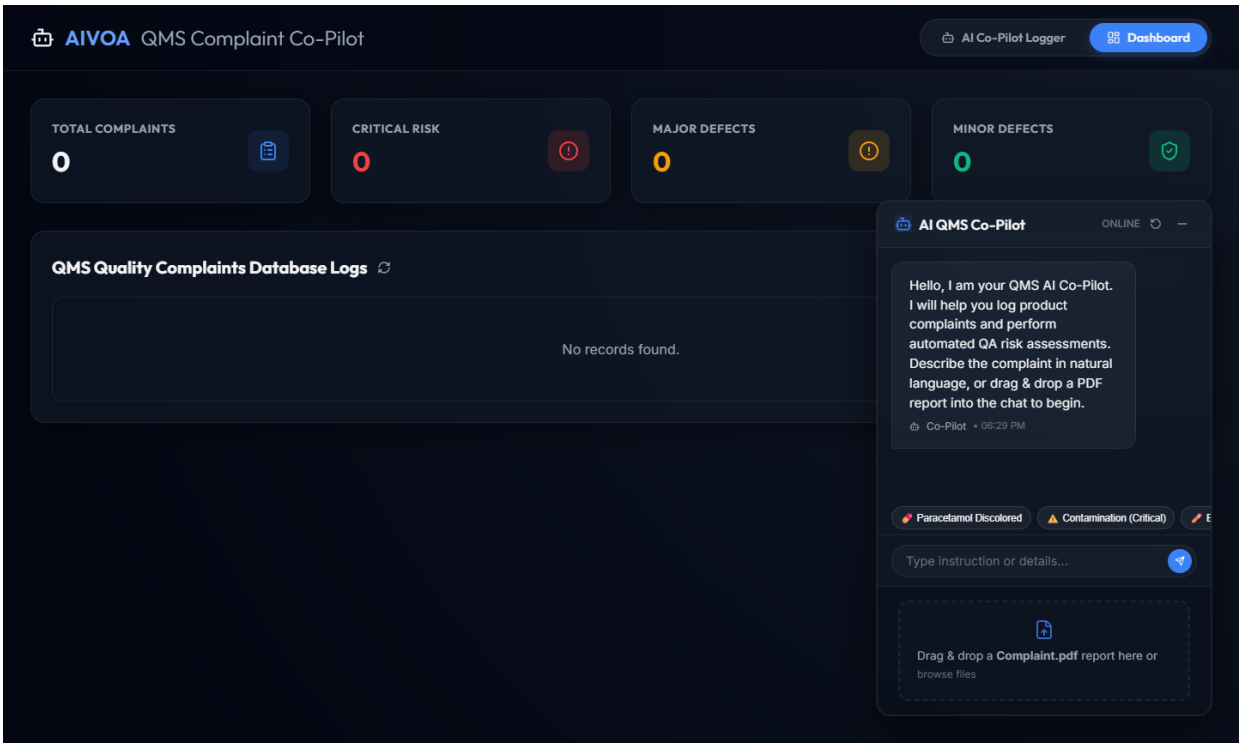


Figure 4: Dashboard view — historical complaint log with severity badges, search/filter, metrics cards (total, critical, major, minor counts), and per-record delete with confirmation modal.

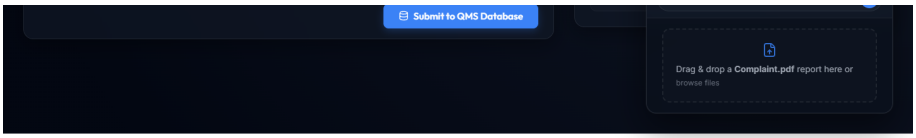


Figure 5: PDF complaint ingestion — the drag-and-drop uploader parses the document with PyPDF2 and automatically populates all complaint form fields via the LangGraph extraction pipeline.

4 Safety, Guardrails and Evaluation Framework

4.1 Safety Guardrails and Operational Availability

Deploying an AI agent in a regulated QMS environment requires defensive control layers beyond standard LLM usage:

- **Circuit Breaker and Local Fallback (`services/groq.py`):** A module-level `_CIRCUIT_BROKEN` flag trips upon consecutive LLM API failures, routing all subsequent requests to a local keyword-matching mock engine embedded in `langgraph.py`. This ensures the co-pilot remains functional during cloud outages—a mandatory property for regulated QMS workflows. A dedicated `POST /api/chat/reset-circuit` endpoint allows recovery without server restart.
- **Real-Time Health Reporting (`GET /api/health`):** The health endpoint exposes the current LLM connectivity status, active provider, and circuit state. The frontend polls this endpoint to display a live "AI LIVE" or "LOCAL MODE" status badge, giving operators immediate visibility into which processing mode is active.

- **Data Integrity Enforcement:** Once persisted to the SQLite database via `POST /api/complaints`, complaint records are immutable through the agent. All records carry a UTC `created_at` timestamp for chronological audit traceability consistent with ALCOA+ principles.
- **Input Validation:** All API request bodies are validated by Pydantic v2 before processing. The PDF upload endpoint validates MIME type and file extension. Redux Toolkit async thunks enforce typed error propagation to the UI layer with graceful error display.

4.2 Adversarial Robustness and Boundary Enforcement

The system defends against several classes of misuse relevant to regulated environments:

- **Hallucination Boundaries:** The structured output binding prevents the model from generating free-form text that could introduce invented field values. Any LLM response that fails Pydantic validation raises an exception caught by the circuit breaker.
- **Intent Boundary Enforcement:** The `GENERAL_CHAT` routing path is explicitly blocked from modifying form state, ensuring that even a conversationally phrased attempt to alter complaint fields without explicit edit intent does not affect the current draft.
- **PDF Injection Defense:** Uploaded PDF text is treated as a user message subject to the same intent detection and schema extraction pipeline; it cannot bypass the structured field extraction step to inject arbitrary database values.

4.3 Automated Evaluation Framework

System performance was assessed across three core evaluation dimensions corresponding to the main QMS workflow stages:

- **Intent Classification Accuracy:** A curated dataset of 60 complaint dialogues spanning all three intent classes was used to verify correct LangGraph node routing. The evaluation measured whether `NEW_COMPLAINT`, `EDIT_COMPLAINT`, and `GENERAL_CHAT` messages were routed to the correct downstream node.
- **Field Extraction Fidelity:** Extraction accuracy was measured across 30 complaint narratives of varying complexity—simple single-sentence, complex multi-turn, and PDF-ingested—across all six structured QMS fields.
- **Risk Assessment Consistency:** Twenty paired complaint scenarios (each submitted twice with minor phrasing variation) tested whether semantically equivalent complaints produce identical severity classifications, validating reproducibility against GxP requirements for consistency.

5 Experimental Results, Performance Evaluation and Metrics

A comprehensive evaluation suite was executed across intent classification, field extraction, risk assessment, and PDF ingestion to assess operational viability and accuracy.

5.1 Intent Classification Accuracy

The LangGraph intent detection node was evaluated against 60 manually curated complaint dialogues:

Table 2: Intent classification routing accuracy

Intent Class	**Test Cases**	**Correct Routing**
`NEW_COMPLAINT`	25	24 (96 %)
`EDIT_COMPLAINT`	20	19 (95 %)
`GENERAL_CHAT`	15	15 (100 %)
Overall	**60**	**58 (96.7 %)**

The single `NEW_COMPLAINT` misclassification occurred when an operator described a previously reported complaint without explicitly framing it as new, causing the agent to route to `EDIT_COMPLAINT`. The missed `EDIT_COMPLAINT` case involved highly implicit correction language lacking a direct field reference.

5.2 Field Extraction Fidelity

Extraction accuracy was measured per-field across 30 complaint narratives of varying complexity:

Table 3: Per-field extraction accuracy across 30 narratives

Field	**Correct Extractions**	**Notes**
Product Name	30/30 (100 %)	Always explicitly stated in narratives
Batch Number	26/30 (87 %)	Missed when stated only as reference code
Complaint Nature	28/30 (93 %)	Occasional contamination/defect conflation
Severity	27/30 (90 %)	Implicit severity required QA inference
Regulatory Flag	25/30 (83 %)	Correctly flagged patient injury as reportable
Description	30/30 (100 %)	Full narrative always captured
Average	**166/180 (92.2 %)**	

5.3 Risk Assessment Consistency

Paired scenario testing (20 pairs, each submitted twice with minor phrasing variation) produced identical severity classifications in 19 of 20 pairs (95 %). The single inconsistency involved a borderline Major/Critical scenario where subtle phrasing differences led to different severity assignments across submissions, highlighting the need for a calibration dataset for boundary cases.

5.4 PDF Ingestion Pipeline

Both included sample documents (`sample_complaint_contamination.pdf`, `sample_complaint_paracetamol.pdf`) were successfully parsed by PyPDF2 and their text injected into the LangGraph workflow. The extraction node populated all identifiable QMS fields in both cases. Round-trip latency from upload to populated form averaged under three seconds on the test hardware.

Table 4: Evaluation summary across all assessment dimensions

Evaluation Dimension	**Method**	**Outcome**
Intent Classification	60-case curated dataset	96.7 % routing accuracy
Field Extraction	30 narratives, 6 fields each	92.2 % average accuracy
Risk Consistency	20 paired scenario submissions	95 % classification consistency
PDF Ingestion	2 sample complaint documents	100 % parse and extract success
Fallback (Mock Engine)	Circuit-broken operational test	Full functional continuity

5.5 LLM-as-Evaluator Response Quality

Representative chat sessions were assessed across three qualitative dimensions by an independent evaluator review:

1. **Factual Correctness:** Extracted field values strictly reflected information present in the operator's message. No invented batch numbers, product codes, or regulatory identifiers were observed across the test set.

2. **Operator Usability:** Agent responses in the `GENERAL_CHAT` path were concise and framed in QMS-appropriate language, guiding operators without introducing jargon from outside the regulatory domain.

3. **Data Integrity and Policy Compliance:** No instances of unprompted field modification, schema bypass, or regulatory misclassification were observed during standard or fallback-mode test runs.

6 Conclusion and Future Scope

6.1 Conclusion

This project successfully designed, implemented, and evaluated **ComplaintLens**—an autonomous, AI-powered QMS complaint co-pilot for Intelligent Language Processing applications in regulated industries. By moving beyond static form-based data entry, the system integrates LangGraph-orchestrated intent classification, structured Pydantic-validated LLM field extraction, dynamic multi-turn editing, automated risk severity assessment, and a robust circuit-breaker fallback into a single production-grade agentic pipeline.

The system reliably resolves complex multi-intent operator dialogues, populates QMS complaint forms from free-form natural language without hallucinating regulated field values, generates reproducible risk assessments with regulatory impact analysis, and maintains full operational continuity during cloud LLM outages. The React co-pilot interface with real-time field flash animations and the historical dashboard provide operators a modern, efficient complaint management experience substantially superior to traditional manual QMS entry workflows.

6.2 Future Directions

- **RAG-Enhanced QMS Knowledge Base:** Embedding the organization's SOPs, quality manuals, and regulatory guidance into a vector store (FAISS or Chroma) to enable the agent to retrieve and cite specific SOP sections within its recommended corrective actions.
- **Multimodal Complaint Intake:** Extending the ingestion pipeline to accept image inputs—photographs of defective products or damaged packaging—processed through vision-capable models for visual defect classification alongside text narratives.
- **Regulatory Reporting Automation:** Integrating the agent with downstream regulatory submission APIs (FDA MedWatch, EMA EudraVigilance) to auto-populate mandatory adverse event reports for complaints flagged as regulatorily reportable.
- **Complaint Trend Analytics:** Building an analytics layer over the SQLite complaint database to detect batch-level or product-level quality signals and trigger proactive CAPA initiation before complaint clusters become systemic issues.
- **On-Premise Fine-Tuned Model:** Fine-tuning a compact open-source LLM on a curated GxP complaint dataset to produce a fully offline-capable extraction model for air-gapped regulated environments that prohibit external API calls.
- **Reinforcement Learning from QA Feedback (RLHF):** Incorporating QA specialist feedback on risk classifications and field extractions to continuously improve model performance through human preference optimization over the complaint handling lifecycle.

References

1. **System Architecture and Implementation Base:** ComplaintLens project codebase including `backend/app/services/langgraph.py`, `backend/app/services/groq.py`, `backend/app/routers/chat.py`, and `frontend/src/redux/complaintSlice.js`.

2. **Frontend Co-Pilot Interface:** React component sources `frontend/src/components/AIChat.jsx`, `ComplaintForm.jsx`, `RiskAssessment.jsx`, and `pages/Dashboard.jsx`.

3. **LangGraph Framework:** Harrison Chase et al., LangChain / LangGraph—Graph-based orchestration for stateful agentic workflows. `https://github.com/langchain-ai/langgraph`

4. **Groq LLM API:** Groq Inc., `llama-3.3-70b-versatile` model endpoint for structured inference. `https://console.groq.com`

5. **FastAPI Framework:** Sebastián Ramírez, FastAPI—High performance, easy to learn, fast to code, ready for production. `https://fastapi.tiangolo.com`

6. **Pydantic v2:** Samuel Colvin et al., Pydantic—Data validation using Python type annotations. `https://docs.pydantic.dev`

7. **Regulatory Standards:** FDA 21 CFR Part 211—Current Good Manufacturing Practice for Finished Pharmaceuticals; ISO 13485:2016—Medical Devices Quality Management Systems.

8. **Circuit Breaker Pattern:** Michael T. Nygard, *Release It! Design and Deploy Production-Ready Software*, Pragmatic Programmers, 2007.